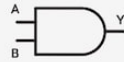


BASIC LOGIC GATES

AND



$$Y = A \cdot B$$

A	0	0	1	1
B	0	1	0	1
Y	0	0	0	1

The output is 1 only when all inputs are 1; otherwise, it's 0.

OR



$$Y = A + B$$

A	0	0	1	1
B	0	1	0	1
Y	0	1	1	1

The output is 1 when any input is 1; otherwise, it's 0.

XOR

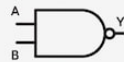


$$Y = A \oplus B$$

A	0	0	1	1
B	0	1	0	1
Y	0	1	1	0

The output is 1 when exactly one input is 1; otherwise, it's 0.

NAND



$$Y = \overline{A \cdot B}$$

A	0	0	1	1
B	0	1	0	1
Y	1	1	1	0

The output is 0 only when all inputs are 1; otherwise, it's 1.

NOR



$$Y = \overline{A + B}$$

A	0	0	1	1
B	0	1	0	1
Y	1	0	0	0

The output is 1 only when all inputs are 0; otherwise, it's 0.

XNOR



$$Y = \overline{A \oplus B}$$

A	0	0	1	1
B	0	1	0	1
Y	1	0	0	1

The output is 1 when all inputs are the same; otherwise, it's 0.

BUFFER



$$Y = A$$

A	0	1
Y	0	1

The output is the same as the input.

NOT



$$Y = \bar{A}$$

A	0	1
Y	1	0

The output is the inverse of the input.

1 MOVE / DATA TRANSFER (ÚPLNÝ ZÁKLAD)

irmovq V, rB

➡ immediate → register

asm

Copy

```
irmovq $10, %rax    # %rax = 10
```

rrmovq rA, rB

➡ register → register

asm

Copy

```
rrmovq %rax, %rbx    # %rbx = %rax
```

rmmovq rA, D(rB)

➡ register → memory

asm

Copy

```
rmmovq %rax, 8(%rbx) # M[%rbx+8] = %rax
```

mrmovq D(rB), rA

➡ memory → register

asm

Copy

```
mrmovq 8(%rbx), %rax # %rax = M[%rbx+8]
```

2 ARITHMETIC / LOGIC (ČÁSTÉ)

addq rA, rB

➡ $rB = rB + rA$

asm

Copy

```
addq %rax, %rbx
```

subq rA, rB

➡ $rB = rB - rA$

asm

Copy

```
subq %rax, %rbx
```

subq rA, rB

 **rB = rB - rA**

asm

subq %rax, %rbx

Copy

andq rA, rB

 **bitwise AND**

asm

andq %rax, %rbx

Copy

xorq rA, rB

 **bitwise XOR**

asm

xorq %rax, %rbx

Copy

 Tieto nastavujú condition codes (ZF, SF, OF)

 CONTROL FLOW (SKÚŠKOVÝ HIT)

jmp Dest

 **unconditional jump**

Conditional jumps (VELMI ČASTÉ)

Instruction	Meaning
j e	equal (ZF = 1)
j ne	not equal
j g	greater
j ge	greater or equal
j l	less
j le	less or equal

asm

j e done

Copy

4 STACK OPERATIONS (URČITE NA SKÚŠKE)

pushq rA

→ push register on stack

```
asm
pushq %rax
```

Copy

Equivalent:

```
code

%rsp = %rsp - 8
M[%rsp] = %rax
```

Copy

popq rA

→ pop from stack

```
asm
popq %rax
```

Copy

Equivalent:

```
code

%rax = M[%rsp]
%rsp = %rsp + 8
```

Copy

5 CALL / RETURN (MUSÍŠ POZNAŤ)

call Dest

→ push return address + jump

```
asm
call func
```

Copy

Equivalent:

```
code

push PC+1
PC = Dest
```

Copy

ret

→ pop return address

```
asm
ret
```

Copy

code

PROCESS	FILE		PU	PT
	SU	ST		
SU	rw	r	r	r
ST	w	rw	-	r
PU	w	-	rw	r
PT	w	w	w	rw